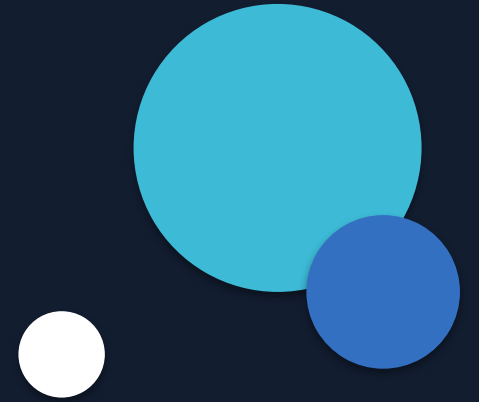


JARVIS Vision Assistant

A Computer Vision Object-Detection Application



Samantha Mireles

ITAI 1378 - Computer Vision and AI

Tier 1 | Individual Project

The Problem

Why this application matters

Real-World Problem

People may need a fast way to identify and keep track of common objects in their surroundings. Manually scanning a busy scene can be slow or inconvenient.

Who Cares?

Users of AI assistants, smart-home systems, accessibility tools, and automated monitoring systems can benefit from software that understands visual scenes.

What Will Solve It?

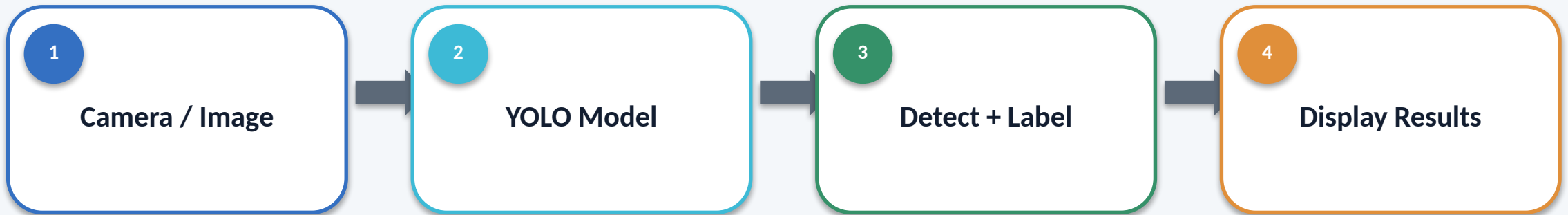
A computer vision assistant that analyzes an image, detects common objects, labels them, and reports what it sees.

Goal: turn a normal image into useful information about the objects inside it.

Your Solution

JARVIS Vision Assistant

A vision-based AI assistant that takes an image, detects common objects, and displays labeled results.



Example output: "Detected: 1 person, 1 laptop, 2 chairs, and 1 cell phone."

Technical Approach

How the system will work

CV Technique

Object Detection - finds multiple objects and their locations within the same image.

Framework / Tools

Python, Ultralytics YOLO, OpenCV, Google Colab, and GitHub.

Model

A pretrained YOLO object-detection model.

Why YOLO?

It is designed for fast object detection and can recognize many common object categories without training a model from scratch.

TIER 1

Pretrained Model

The focus is building a complete, useful application around an existing model.

Data Plan

Reachable, realistic, and appropriate for Tier 1

Source

Student-collected photos plus appropriate public/sample images.

Rough Size

Approximately 30-50 test images containing everyday objects.

Labels

Labels come from the pretrained YOLO model rather than a custom training dataset.

Example object categories

person

chair

laptop

cell phone

bottle

backpack

car

The project tests an application; it does not require building a large dataset.

Success Metrics

How I will know the project works

PRIMARY METRIC

$\geq 85\%$

of clearly visible supported objects in the selected test set are correctly identified.

SECONDARY METRIC

$< 2 \text{ sec}$

to process each image in the main testing environment.

Measurement: compare detections with visible objects and record processing time during testing.

Milestone Plan

16-week term roadmap



Blueprint

Plan approved

Week 10



First Working Demo

YOLO runs end-to-end

Week 11



Make It Yours

My images + app logic

Weeks 12-13



Improve & Measure

Test + record metrics

Week 14



Package & Present

Demo + README + slides

Week 15

First prove the pipeline works; then invest time in data, customization, and measurement.

Risks & Resources

Plan B keeps the project achievable

Risk 1 - Missed or incorrect detections

Plan B: use clearer images, better lighting, supported object categories, and adjust confidence settings when appropriate.

Compute & Software

Google Colab, Python, Ultralytics YOLO, OpenCV, and GitHub.

Risk 2 - Real-time camera is too slow or complex

Plan B: use uploaded still images. The full input → model → output pipeline will still be demonstrated.

Estimated Cost

\$0 planned by using free/open-source software and free compute resources where available.

Final goal: a small, working proof of concept that can identify everyday objects and report what it sees.